Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 5

з дисципліни «Теорія прийняття рішень»

Тема: «Методи експертного оцінювання»

Виконав:
студент групи ФЕІ-34с
Біжик Юрій Володимирович

Перевірила:
доцент Дуфанець Марта
Василівна

Львів — 2026

1 Мета та постановка задачі

Засвоїти методи експертного оцінювання. Виконати безпосереднє числове оцінювання, метод середньої точки і метод Черчмена–Акоффа аналітично, а для методу Терстоуна розробити програму визначення математичних сподівань альтернатив.

Набір альтернатив від викладача не надано. Тому всі обчислення нижче стосуються явно навчального набору: п'ять умовних конфігурацій ноутбука для навчання. Оцінки описують особисту модель переваг, а не результати опитування фахівців; умовні ймовірності також не є частотами реального експерименту. Для цього завдання номер у журналі не впливає на кількість альтернатив.

Альтернатива	Зміст навчальної альтернативи
A1	Продуктивний ноутбук: пріоритет швидкодії та роботи з великими задачами
A2	Збалансований ноутбук для більшості навчальних задач
A3	Портативний ноутбук із помірною продуктивністю
A4	Базовий ноутбук для простих завдань
A5	Мінімальна конфігурація з найбільшими обмеженнями

Навчальне початкове ранжування: $A_1 > A_2 > A_3 > A_4 > A_5$. Ці назви не позначають конкретні товарні моделі; реальні ціни та характеристики до оцінювання не залучалися.

2 Безпосереднє числове оцінювання

Експерт безпосередньо призначає бали на шкалі від 0 до 100. Для навчального сценарію прийнято $f = (90, 75, 60, 45, 30)$. Отже, $A_1 > A_2 > A_3 > A_4 > A_5$, а найкраща альтернатива — A1. Різниця між послідовними оцінками становить 15 балів.

Ці бали використовуються як інтервальна шкала: різниці мають зміст лише за прийнятої моделі експерта. Звідси не випливає, що A1 утричі краща за A5: для такого висновку потрібна шкала відношень із змістовним нулем.

3 Метод середньої точки

Визначаю найгіршу альтернативу A5 і найкращу A1 та задаю їм оцінки $f(A_5) = 0$, $f(A_1) = 1$. Нуль тут є довільним якорем, а не

відсутністю корисності. За навчальним судженням експерта A3 лежить посередині:

$$f(A_3) = \frac{f(A_5) + f(A_1)}{2} = 0.5. \tag{1}$$

Наступний поділ відрізків дає:

$$f(A_4) = \frac{f(A_5) + f(A_3)}{2} = 0.25, \tag{2}$$

$$f(A_2) = \frac{f(A_3) + f(A_1)}{2} = 0.75. \tag{3}$$

Отримано $f = (1, 0.75, 0.5, 0.25, 0)$. Метод зберіг початкове ранжування. Збіг із безпосередніми оцінками пояснюється конструкцією навчальних суджень: перетворення $\frac{f_{\text{direct}} - 30}{60}$ дає саме середньоточкові оцінки. У реальному опитуванні рівновіддалені альтернативи можуть бути відсутні; їх не можна оголошувати середніми лише за порядковим номером.

4 Метод Черчмена–Акоффа

Метод передбачає невід'ємні оцінки, збереження ранжування та адитивність сумісної реалізації альтернатив. Для взаємовиключних покупок ноутбука адитивність не є фізичною властивістю: тут це лише умовна навчальна модель порівняння внесків. Її не можна переносити на реальну покупку без обґрунтування.

Приймаю початкові оцінки $(1, 0.5, 0.3, 0.15, 0.05)$ і навчальні судження, що кожна альтернатива переважає суму всіх гірших. Для A1 початкова сума дорівнює $0.5 + 0.3 + 0.15 + 0.05 = 1$, тому судження про строгу перевагу не відображене. Для A2 також $0.3 + 0.15 + 0.05 = 0.5$, тобто початкові оцінки дають рівність.

Корегую A2 до 0.55, а після цього A1 до 1.10. Число 1 не є жорсткою верхньою межею: методичка вимагає $f(A_i) \geq 0$, а одиницю найкращій альтернативі призначають лише інколи. Остаточна шкала:

$$f = (1.10, 0.55, 0.30, 0.15, 0.05). \tag{4}$$

Порівняння	Числовий зміст	Судження
A1 і A2+A3+A4+A5	$1.10 > 1.05$	Виконується
A2 і A3+A4+A5	$0.55 > 0.50$	Виконується

Порівняння	Числовий зміст	Судження
A3 і A4+A5	$0.30 > 0.20$	Виконується
A4 і A5	$0.15 > 0.05$	Виконується

Оскільки на кожному етапі альтернатива переважає суму всіх наступних, гілка послідовного зменшення числа доданків не потрібна. Порядок строго збережений. Загальна сума оцінок — 2.15; за потреби можна поділити всі оцінки на 2.15, і кожна нерівність залишиться чинною. Додавання константи, навпаки, не зберігає таких порівнянь: адитивний метод вимагає шкали відношень, а не довільної інтервальної шкали.

5 Програмна реалізація методу Терстоуна

Приймаю випадок V: незалежні нормальні оцінки, рівні дисперсії та одиницю вимірювання $\sqrt{2D}$. Для ймовірності переваги $p_{ij} = P\big(f(A_i) > f\big(A_j\big)\big)$ використовується квантиль:

$$z_{ij} = \Phi^{-1}\big(p_{ij}\big), \quad M_i - M_j = z_{ij}. \tag{5}$$

Для незалежних оцінок дисперсія різниці дорівнює $D_i + D_j$. У загальному корельованому випадку вона дорівнює $D_i + D_j - 2\rho_{ij}\sqrt{D_i D_j}$; друкарська суперечність знака у теоретичному тексті не впливає на обраний випадок нульової кореляції.

Для несумісних парних суджень мінімізується

$$S = \sum_{i<j} \big(M_i - M_j - z_{ij}\big)^2, \quad \sum_i M_i = 0. \tag{6}$$

Для повної взаємодоповнювальної матриці $z_{ji} = -z_{ij}$. Нормальні рівняння дають $nM_i - \sum_j M_j = \sum_j z_{ij}$, тому $M_i = \big(\tfrac{1}{n}\big)\sum_j z_{ij}$. Це МНК-розв’язок, а не евристичне усереднення. Нульова сума усуває невизначеність спільного зсуву шкали.

Для демонстрації синтетичні ймовірності побудовано за правилом $p_{ij} = \Phi(j - i)$, де індекси зростають від A1 до A5. Для різниць 1, 2, 3, 4 це приблизно 0.841345, 0.977250, 0.998650, 0.999968. Обернені ймовірності дорівнюють $1 - p_{ij}$, а діагональ — 0.5. Такий набір спеціально узгоджений; він ілюструє метод, але не демонструє розбіжностей між реальними експертами.

Аналітично $z_{ij} = j - i$ і $M = (2, 1, 0, -1, -2)$. Після зсуву мінімуму до нуля: $M_{\text{shifted}} = (4, 3, 2, 1, 0)$. Теоретична сума квадратів

нев'язок дорівнює нулю; програмний результат може містити малу похибку числового округлення. Після запуску розробленої програми отримано центровані оцінки 2; 1; 0; –1; –2, а після зсуву мінімуму до нуля — 4; 3; 2; 1; 0. Фактична сума квадратів нев'язок становить 0.00000000000000000000000003578482607260459 відмінність від теоретичного нуля пояснюється округленням синтетичних ймовірностей та обчисленням квантилів. Значення підставлені зі створеного програмою файла `results.json`.

Програма `main.py` читає `input.json`, застосовує `NormalDist.inv_cdf`, обчислює центровані та зсунуті оцінки, ранжування й суму квадратів нев'язок, виводить таблицю та зберігає `results.json`. Значення 0 і 1 не перетворюються довільно на скінченні квантилі: для таких частот потрібна явно погоджена поправка з урахуванням кількості порівнянь.

6 Порівняння результатів

Альт.	Бали	Середня точка	Черчмен–Акофф	Терстоун
A1	90	1.00	1.10	2
A2	75	0.75	0.55	1
A3	60	0.50	0.30	0
A4	45	0.25	0.15	–1
A5	30	0.00	0.05	–2

Порівнювати абсолютні числа між колонками некоректно: вони мають різні якорі та припущення. Натомість усі чотири методи дають однакове ранжування навчального набору. У Черчмена–Акоффа інтервали нерівні, бо оцінки узгоджено з перевагою над сумою гірших альтернатив.

7 Висновки

Для обраного мною навчального сценарію A1 є найкращою альтернативою в усіх методах. Я розмежував безпосередню бальну оцінку, рекурсивний поділ шкали та адитивні судження, а також вивів МНК-оцінки Терстоуна. Головне обмеження результату — синтетичні судження: для практичного вибору ноутбука потрібні дані про конкретні моделі та реальні переваги експертів.

Метод Черчмена–Акоффа потребує окремого обґрунтування адитивності, а Терстоун — достатньої кількості парних порівнянь та прийнятності припущень нормальної моделі.

8 Використані джерела

1. Методичні вказівки до лабораторної роботи № 5 «Методи експертного оцінювання», файл `Lab5.pdf`.
2. Катренко А. В., Пасічник В. В., Пасько В. П. Теорія прийняття рішень. Київ: BHV, 2009. 448 с.

9 Додаток. Програмний код

```
"""Метод Терстоуна, випадок V: повна матриця парних переваг."""

import argparse
import json
import math
from pathlib import Path
from statistics import NormalDist


def thurstone(probabilities):
    """МНК для z_ij = M_i - M_j з обмеженням sum(M_i) = 0."""
    n = len(probabilities)
    if n < 2 or any(len(row) != n for row in probabilities):
        raise ValueError("Потрібна квадратна матриця щонайменше 2×2.")
    normal = NormalDist()
    z = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            p = probabilities[i][j]
            if not isinstance(p, (int, float)) or not math.isfinite(p):
                raise ValueError("Ймовірності мають бути скінченними
числами.")
            if i == j:
                if not math.isclose(p, 0.5, abs_tol=1e-9):
                    raise ValueError("Діагональні ймовірності мають
дорівнювати 0.5.")
            else:
                if not 0 < p < 1:
                    raise ValueError("Для скінченних квантилів потрібно 0 <
p < 1.")
                if not math.isclose(p + probabilities[j][i], 1,
abs_tol=1e-9):
                    raise ValueError("Потрібно p_ij + p_ji = 1.")
                z[i][j] = normal.inv_cdf(p)
    means = [sum(row) / n for row in z]
    # Формула є розв'язком нормальних рівнянь для повного графа порівнянь.
    residuals = [means[i] - means[j] - z[i][j]
                 for i in range(n) for j in range(i + 1, n)]
    sse = sum(r * r for r in residuals)
```

```
    return {"quantiles": z, "means": means,
            "shifted_means": [m - min(means) for m in means],
            "sse": sse, "rmse": math.sqrt(sse / len(residuals)),
            "ranking": sorted(range(n), key=lambda i: (-means[i], i))}


def solve(data):
    names = data["alternatives"]
    result = thurstone(data["probabilities"])
    n = len(names)
    if len(result["means"]) != n:
        raise ValueError("Кількість альтернатив і розмір матриці різні.")
    for key in ("direct_scores", "midpoint_scores", "churchman_initial",
"churchman_final"):
        if len(data[key]) != n or any(not math.isfinite(x) for x in
data[key]):
            raise ValueError(f"Некоректні оцінки {key}.")
    final = data["churchman_final"]
    if any(x < 0 for x in final) or any(final[i] <= final[i + 1] for i in
range(n - 1)):
        raise ValueError("Оцінки Черчмена мають бути невід'ємними і строго
спадати.")
    comparisons = []
    for judgement in data["churchman_judgements"]:
        i, group = judgement["left"], judgement["right"]
        if i < 0 or i >= n or not group or i in group or len(set(group)) !=
len(group):
            raise ValueError("Некоректний набір індексів порівняння.")
        if any(j < 0 or j >= n for j in group):
            raise ValueError("Індекс альтернативи поза діапазоном.")
        a, b = final[i], sum(final[j] for j in group)
        relation = judgement["relation"]
        if relation not in (">", "<="):
            raise ValueError("Відношення має бути > або <=.")
        satisfied = a > b if relation == ">" else a <= b
        comparisons.append({**judgement, "left_score": a, "right_sum": b,
                            "satisfied": satisfied})
    return {"note": data["note"], "alternatives": names,
            "direct_scores": data["direct_scores"],
            "midpoint_scores": data["midpoint_scores"],
            "churchman_scores": final, "churchman_comparisons":
comparisons,
            "thurstone": result,
            "ranking": [names[i] for i in result["ranking"]]}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path,
default=Path(__file__).with_name("input.json"))
    parser.add_argument("--output", type=Path,
default=Path(__file__).with_name("results.json"))
    args = parser.parse_args()
    try:
        result = solve(json.loads(args.input.read_text(encoding="utf-8")))
```

```
        args.output.write_text(json.dumps(result, ensure_ascii=False,
indent=2) + "\n", encoding="utf-8")
    except (ValueError, KeyError, TypeError, OSError) as exc:
        parser.exit(2, f"Помилка вхідних даних: {exc}\n")
    print(result["note"])
    print("Альтернатива | M (центрована шкала) | M (мінімум = 0)")
    for i in result["thurstone"]["ranking"]:
        print(f'{result["alternatives"][i]} | {result["thurstone"]["means"]
[i]:.6f} | '
              f'{result["thurstone"]["shifted_means"][i]:.6f}')
    print(f'Сума квадратів нев’язок: {result["thurstone"]["sse"]:.8g}')


if __name__ == "__main__":
    main()
```